



apps4all

Crear, añadir, mejorar

Tu web, paso a paso

Guía para poner en marcha **apps4all.app** sin saber nada de dominios, repositorios ni servidores. Todo explicado desde cero, y con el porqué de cada decisión.

Lo primero: esto es normal

Ayer terminaste la sesión con la sensación de que esto era más lío de lo esperado. Es una reacción correcta y conviene entender por qué se produce, porque explica también por qué se pasa.

Montar una web tiene dos partes muy distintas. La primera es **la instalación**: comprar la dirección, contratar el sitio donde vivirá, activar el candado de seguridad, preparar el archivo de versiones. Es aburrida, está llena de palabras raras y hay que hacerla **una sola vez en la vida del proyecto**. La segunda parte es **hacer la web**: escribir, maquetar, enseñar tus trabajos. Esa es divertida, es la que sabes hacer y es la que ocupará el 95 % del tiempo a partir de la semana que viene.

Ayer estabas de lleno en la primera parte. Toda la jerga que apareció —dominio, DNS, SSL, FTP, repositorio, commit— es de instalación. En cuanto esté hecha, la mayoría de esas palabras no vuelven a aparecer.

LA COMPARACIÓN QUE LO ORDENA TODO

Es exactamente igual que abrir un local. Antes de vender nada hay que firmar el alquiler, dar de alta la luz, poner el rótulo y conseguir las llaves. Nadie que abre una tienda dice «esto de la tienda es complicadísimo» por el papeleo del primer día: sabe que es de una vez. Aquí igual.

Hay además una razón concreta por la que ayer viste tantos pasos juntos: te preparé el procedimiento *completo*, incluido el despliegue automático, que en realidad es opcional y va después. En esta guía está ordenado por urgencia, y lo que no es urgente está marcado como tal.

Cómo usar este documento

No hace falta leerlo entero de una sentada ni memorizar nada.

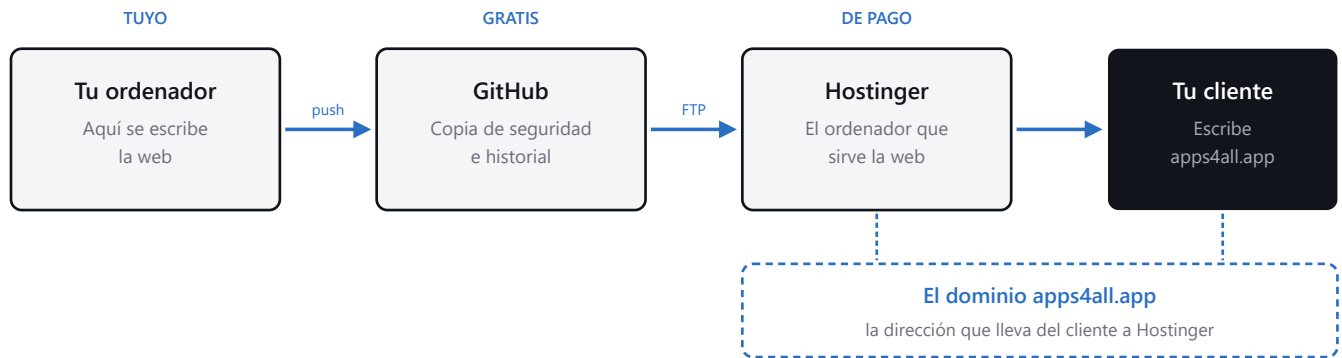
- **Capítulos 2 y 3** — el mapa y el diccionario. Léelos con calma; son los que hacen que el resto se entienda.
- **Capítulo 6** — lo que te toca hacer. Ese sí, con el ordenador delante, tachando casillas.
- **Capítulos 8 y 9** — el porqué de las decisiones y hacia dónde crece esto. Para cuando quieras discutirlos, que es sano.
- **Capítulos 10 y 11** — para consultar cuando algo falle o no recuerdes una orden.

SI SOLO LEES UNA COSA

Del capítulo 6, haz las tareas **1, 2 y 3**. Con esas tres, la próxima sesión ya podemos publicar algo. Las otras tres se pueden hacer sobre la marcha.

El mapa completo

Una web en funcionamiento son cuatro piezas separadas que se contratan por separado y que hay que conectar entre sí. Casi toda la confusión inicial viene de no saber cuántas piezas hay ni de quién es cada una.



El viaje de un cambio: de tu teclado a la pantalla de quien te busca.

Pieza por pieza

Pieza	Qué es en realidad	La comparación
El dominio apps4all.app	El nombre que se teclea en el navegador. Se alquila por años.	La dirección postal de tu negocio. No es el local: es el papel donde pone dónde está el local.
El hosting Hostinger	Un ordenador encendido siempre en un centro de datos, que guarda tus archivos y se los manda a quien los pida.	El local alquilado . Es donde está de verdad la mercancía.
Git y GitHub	Un sistema que guarda todas las versiones por las que ha pasado tu web, y una copia en internet de ese historial.	El archivo de la empresa , con una copia en el banco. Nada se pierde nunca, y siempre puedes volver a como estaba en marzo.
Los archivos HTML y CSS	Los documentos que forman la web. HTML es el contenido; CSS es el aspecto.	El género y la decoración del local. Lo único que verá tu cliente.

EL MALENTENDIDO MÁS COMÚN DE TODOS

El dominio y el hosting son cosas distintas y se pagan por separado, aunque los vendan juntos. Se puede tener dominio sin hosting (una dirección sin local) y hosting sin dominio (un local sin rótulo). Que los dos estén en Hostinger es una comodidad, no una obligación — y por eso la primera tarea del capítulo 6 es averiguar dónde está cada uno.

Y GitHub, ¿para qué, si la web ya está en Hostinger?

Es la pregunta razonable. La web funciona perfectamente sin GitHub. Se usa por tres motivos, y ninguno es «porque es lo que se hace»:

- **Deshacer.** Cambias algo, queda peor, y no recuerdas cómo estaba. Con GitHub vuelves a la versión de anteayer en diez segundos. Sin GitHub, lo reconstruyes de memoria.
- **Un servidor no es una copia de seguridad.** Si borras un archivo en Hostinger, está borrado. Si lo borras teniéndolo en GitHub, sigue estando en GitHub.
- **Publicar sin pensar.** Con el automatismo del capítulo 7, guardar un cambio y publicarlo es la misma acción. Sin él, cada cambio son cinco minutos de comprimir, subir y descomprimir — y esos cinco minutos son justo lo que hace que uno acabe no actualizando nunca la web.

Las palabras nuevas, en cristiano

Todo lo que apareció ayer, sin dar nada por sabido. No hay que aprendérselo: está aquí para volver cuando una palabra no suene.

Del mundo de las direcciones

Dominio	El nombre que se escribe en el navegador. Se alquila por años a un registrador; si dejas de pagarlo, lo pierdes. Tu número de teléfono: es tuyo mientras pagues la línea.
TLD <i>top-level domain</i>	La terminación: .com, .es, .app. Cada una tiene su dueño y sus normas propias. La del .app es la que nos afecta (capítulo 4).
Registrador	La empresa a la que le alquilas el dominio: Hostinger, GoDaddy, Namecheap... Puede ser distinta de la del hosting, y ahí empieza casi todo el lío.
DNS	El sistema mundial que traduce apps4a11.app al número del ordenador donde está tu web. Las páginas amarillas: buscas por nombre y te dan la dirección.
Nameservers <i>servidores de nombres</i>	El apunte que dice qué empresa manda sobre el DNS de tu dominio. Si compraste el dominio fuera de Hostinger, hay que cambiarlos para que apunten allí. Un cambio de domicilio en Correos: dices quién gestiona ahora tu correspondencia.
Propagación	El tiempo que tardan los DNS del mundo en enterarse de un cambio. De minutos a 24 horas. No es un error, es espera , y por eso los cambios de dominio se hacen lo primero.

Del mundo de la seguridad

HTTPS	La versión cifrada de la conexión. La s es de <i>segura</i> : nadie por el camino puede leer ni cambiar lo que se envía.
Certificado SSL	El archivo que le permite al servidor hablar en HTTPS. Hostinger lo da gratis y se activa con un botón; caduca y se renueva solo. El carné que hay que enseñar en la puerta. Sin él no se entra.

HSTS	Una norma que dice «este dominio <i>solo</i> se sirve por HTTPS, sin excepciones». Los navegadores traen una lista de dominios así grabada de fábrica. Todo .app está en esa lista , y eso tiene una consecuencia práctica muy fuerte — capítulo 4.
------	--

Del mundo del hosting

hPanel	El panel de control de Hostinger. La web donde se administra todo: dominios, correos, SSL, archivos.
public_html	La carpeta del servidor cuyo contenido se publica en internet. Lo que metas ahí se ve; lo que dejes fuera, no. El escaparate. El almacén de atrás existe, pero no se ve desde la calle.
index.html	Nombre reservado: es el archivo que el servidor enseña cuando alguien entra en una carpeta sin pedir nada concreto. Es la portada.
FTP	El protocolo de toda la vida para copiar archivos a un servidor. Tiene servidor, usuario y contraseña propios, distintos de los de tu cuenta de Hostinger. La furgoneta de reparto y su llave del almacén.

Del mundo del código

HTML	El lenguaje del contenido : esto es un título, esto un párrafo, esto una imagen. Se lee bastante bien aunque no lo hayas visto nunca.
CSS	El lenguaje del aspecto : los títulos en azul, márgenes de tanto, esta caja a la derecha. Un solo archivo CSS manda sobre todas las páginas a la vez.
Sitio estático	Una web hecha de archivos ya terminados, que el servidor entrega tal cual. No hay base de datos ni programa ejecutándose detrás. Es lo que vamos a hacer: rapidísimo, seguro y casi imposible de romper.
Framework	Un programa que te construye la web a partir de piezas, para no repetir lo mismo en veinte archivos. Astro es el que usaremos algún día. Hoy no hace falta (capítulo 8).
Node y npm	El motor que necesitan esos frameworks para funcionar, y su instalador de piezas. En tu ordenador no están instalados , y ese dato ha decidido buena parte del plan.

Del mundo de Git

Git	Un programa <i>en tu ordenador</i> que va guardando fotos del proyecto entero cada vez que se lo pides. Ya lo tienes instalado.
GitHub	Una web donde se guarda una copia de ese historial. Git y GitHub no son lo mismo : Git es el programa, GitHub es el sitio. Se puede usar Git sin GitHub.
Repositorio <i>repo</i>	La carpeta del proyecto con todo su historial dentro. Cuando dices «crear un repositorio», dices «empezar a llevar la cuenta de los cambios de esta carpeta».
Commit	Una de esas fotos, con una nota tuya explicando qué cambiaste. Es la unidad de trabajo: se hacen muchos al día. Un asiento contable: fecha, concepto y qué se movió. Y por eso el concepto tiene que ser bueno.
Push	Mandar a GitHub los commits que has hecho en tu ordenador. Hasta que no haces push, tu trabajo solo existe en tu disco duro.
Rama <i>branch</i>	Una línea de trabajo paralela para probar algo sin tocar lo que está publicado. La principal se llama <code>main</code> . Se prueba en una rama y, si va bien, se junta con <code>main</code> .
Issue	Una nota abierta en GitHub sobre algo pendiente. Trabajando solo parece innecesario, y es justo al revés: es donde estará escrito <i>por qué</i> dejaste algo a medias cuando vuelvas dentro de un mes.
GitHub Actions	Un robot gratuito de GitHub que hace tareas solo cada vez que subes cambios. Lo usaremos para que publique la web sin que tú hagas nada.
Secret	Una contraseña guardada en GitHub en una caja fuerte: el robot puede usarla, pero nadie puede leerla, ni siquiera tú una vez guardada. Ahí es donde irá la contraseña del FTP.

La trampa del .app

Esto merece capítulo propio porque es la única cosa de todo el proyecto capaz de hacerte perder una tarde entera creyendo que has hecho algo mal cuando está todo bien.

EL HECHO

El TLD `.app` es propiedad de Google, y Google lo dio de alta entero en la lista HSTS que los navegadores traen grabada de fábrica. Traducido: `http://apps4all.app` **no carga**. No carga con un aviso de «sitio no seguro» que se pueda saltar — **no llega ni a intentarlo**. Chrome, Firefox, Safari y Edge se niegan a conectar antes siquiera de preguntarle al servidor si hay algo ahí.

Por qué importa tanto en tu caso

Con un dominio `.com`, el orden natural de trabajo es: subo los archivos, veo la web, y ya activaré el candado cuando tenga un rato. Con un `.app` ese orden **no funciona**, y falla de la peor manera posible: no da un error que te oriente, da una página en blanco. Y ante una página en blanco, lo primero que uno piensa es «he subido mal los archivos», y se pasa dos horas subiéndolos otra vez.

LA CONSECUENCIA PRÁCTICA

El certificado SSL se activa **antes** de subir el primer archivo. Por eso en el capítulo 6 el SSL es la tarea 3 y la primera subida no llega hasta después. No es orden caprichoso: es el único orden en el que puedes distinguir un problema real de una espera normal.

¿Fue mala idea elegir un .app?

No, y conviene decirlo porque suena a defecto y no lo es. Que el TLD obligue a HTTPS significa que **tu web nunca podrá viajar sin cifrar**, ni por un descuido de configuración. Para un estudio que desarrolla software es una señal correcta, y además `apps4all.app` se lee bien y dice lo que haces. La única factura que pasa es esta: acordarse de que el candado va primero.

CÓMO COMPROBARLO CUANDO LLEGUE EL MOMENTO

Escribe la dirección en el navegador siempre con `https://` delante, al menos las primeras veces. Y si ves página en blanco, antes de tocar nada, vuelve al hPanel y mira si el SSL pone **Activo**. Nueve de cada diez veces es eso.

Lo que ya está hecho

Antes de pedirte nada, lo que hay hecho a día de hoy. Es poco, pero es real y no hay que rehacerlo.

Qué	Dónde y en qué estado
La identidad de la marca	Hecha anteayer: logotipo en sus cinco versiones, paleta de colores, el claim <i>Crear, añadir, mejorar</i> y las normas de uso. Está guardada como una capacidad permanente de Claude, así que se aplica sola en todo lo que hagamos de apps4all — este PDF incluido.
La carpeta del proyecto	D:\Claude\Code\apps4all-web — creada, con los logotipos y el CSS de marca ya dentro, en assets/. Todavía no hay ni una página web.
El procedimiento técnico	PROCEDIMIENTO.md, en esa misma carpeta. Es la versión corta y técnica de este PDF: sirve de chuleta cuando ya sepas de qué va cada cosa.
Esta guía	En guía/, junto al HTML del que sale. Si algún día hay que corregirla, se corrige el HTML y se vuelve a imprimir.
Las decisiones técnicas	Tomadas y razonadas — capítulo 8. Ninguna es irreversible.

NADA DE ESTO HA COSTADO DINERO NI COMPROMETE A NADA

No se ha contratado ningún servicio, no se ha creado ninguna cuenta y no se ha publicado nada. Todo lo hecho vive en tu disco duro. Si mañana decides cambiar de rumbo entero, no hay que deshacer nada.

Lo que falta, en una frase

Falta conectar las cuatro piezas del capítulo 2 —eso es tu parte, porque son cuentas y contraseñas tuyas— y escribir las páginas, que es la mía.

Lo que te toca a ti

Seis tareas. Ninguna las puedo hacer yo: todas pasan por paneles web con tu sesión iniciada, o por decisiones que son tuyas. Calcula unos 40 minutos en total, más los ratos de espera.

El orden importa en las tres primeras — la 1 puede tener 24 horas de espera detrás y la 3 no funciona sin la 2. De la 4 en adelante, da igual.

1 Averiguar dónde está el dominio

5 min · lo primero

QUÉ ES ESTO

Comprobar si `apps4all.app` está comprado y, sobre todo, *en qué empresa*. Es el dato que decide si podemos publicar mañana o si hay que esperar un día.

POR QUÉ VA PRIMERO

Porque es la única tarea con una espera larga metida dentro. Si el dominio está en otra empresa, hay que apuntarlo a Hostinger y la *propagación* tarda de 1 a 24 horas. Descubrirlo mañana por la tarde sería perder el día; descubrirlo hoy es lanzarlo y seguir con lo demás mientras tanto.

CÓMO SE HACE

Entra en el hPanel de Hostinger y ve a **Dominios**. Busca `apps4all.app` en la lista. Según lo que veas:

Si...	Entonces
Aparece en la lista	Está en Hostinger. Perfecto, no toques nada, pasa a la tarea 2.
No aparece, pero sabes que lo compraste	Está en otro registrador. Busca en tu correo el justificante de compra para saber en cuál. Habrá que cambiar los <i>nameservers</i> ; dímelo y te paso los de Hostinger.
No lo tienes	Cómpralo, preferiblemente en Hostinger: te ahorras el paso de los nameservers y la espera entera. Unos 15-20 € al año.

APUNTA EL RESULTADO

El dominio está en: _____

2 Crear el sitio vacío en Hostinger

5 min

QUÉ ES ESTO

Decirle a Hostinger «reserva sitio para una web que se llamará apps4all.app». Con eso te crea la carpeta `public_html`, que es el escaparate del capítulo 3.

CÓMO SE HACE

hPanel → **Sitios web** → **Añadir sitio web** → elegir **Sitio vacío**, y asociarlo al dominio.

ELIGE «SITIO VACÍO», NO WORDPRESS NI EL CREADOR DE WEBS

Los dos son productos legítimos, pero hacen lo contrario de lo que buscamos: te llenan `public_html` de archivos suyos y montan un sistema entero por encima. Luego hay que limpiarlo, y limpiar mal deja la web medio rota. Vacío significa vacío: la carpeta la llenamos nosotros.

CÓMO SABES QUE HA IDO BIEN

En hPanel → **Administrador de archivos** existe una carpeta `public_html`, y está vacía o casi. Anota su ruta completa, que se parecerá a `/home/u123456789/domains/apps4all.app/public_html`.

Ruta: _____

3 Activar el certificado SSL

5 min + espera

QUÉ ES ESTO

Poner el candado. Es lo que permite que la dirección funcione con `https://`.

POR QUÉ VA ANTES DE SUBIR NADA

Por el capítulo 4: siendo un `.app`, sin candado la web no se ve *en absoluto*. Si subiéramos los archivos primero, verías una página en blanco y no tendrías forma de saber si el problema son los archivos o el certificado.

CÓMO SE HACE

hPanel → **Seguridad** → **SSL** → instalar el certificado gratuito para `apps4all.app`. Activa también **Forzar HTTPS**, que hace que cualquiera que llegue por `http` sea redirigido al `https`.

CÓMO SABES QUE HA IDO BIEN

El panel pone **Activo**. Puede tardar de 5 minutos a una hora; mientras tanto no hay nada que arreglar, solo que esperar.

4 Anotar las credenciales de FTP

5 min · se puede dejar para luego

QUÉ ES ESTO

El usuario y la contraseña con los que se copian archivos al servidor. Son **distintos** de los de tu cuenta de Hostinger, aunque se saquen del mismo panel.

PARA QUÉ LAS QUIERES

Para el automatismo del capítulo 7, que publica la web sola. Si prefieres empezar publicando a mano, esta tarea puede esperar sin problema.

CÓMO SE HACE

hPanel → **Archivos** → **Cuentas FTP**. Anota **servidor**, **usuario** y **puerto**. Si no recuerdas la contraseña, cámbiala ahí mismo: no pasa nada, no rompe nada.

ESTAS TRES COSAS NO ME LAS MANDES POR EL CHAT

Y no es una formalidad. Una contraseña que pasa por una conversación acaba copiada en sitios de los que uno no lleva la cuenta. Tienen un único destino correcto: la caja fuerte de GitHub (los *secrets*), donde el robot las usa sin que nadie pueda volver a leerlas. Yo te guío para ponerlas ahí; tú las tecleas.

La misma regla, más importante todavía: **una contraseña no se escribe nunca dentro de un archivo del proyecto**. Ni «solo un momento», ni «luego la quito». Al entrar en el historial de Git queda ahí para siempre, aunque después borres la línea.

QUÉ ES ESTO

Una cuenta gratuita en `github.com`, donde vivirá la copia del historial de la web.

CÓMO SE HACE

- ☐ Si no tienes cuenta: `github.com/signup`. Correo, usuario y contraseña, nada más.
- ☐ En **Settings** → **Emails**, comprueba que `jcgualde@yahoo.es` está dado de alta. Git en tu ordenador ya está configurado con ese correo, y eso hace que los cambios te aparezcan atribuidos a ti.
- ☐ Anota tu nombre de usuario: _____

RECOMENDADO, NO OBLIGATORIO

Instalar **GitHub CLI**, una herramienta que convierte «crear el repositorio, subirlo y configurarlo» en una sola orden — hoy y en todos los proyectos que vengan. Se instala con esto en una terminal:

```
winget install --id GitHub.cli
```

Y después, en una terminal *nueva* (elige *GitHub.com*, luego *HTTPS*, luego *Login with a web browser*):

```
gh auth login
```

Si prefieres no instalar nada, no pasa nada: en el capítulo 11 está también el camino por la web, que es un par de clics más.

Lo único de toda la lista que no puede salir de una plantilla, y lo único que de verdad diferencia tu web de cualquier otra. Escríbemelo **en sucio**, en cuatro líneas y sin redactar — de redactar me encargo yo.

A) QUÉ TRABAJOS PUEDES ENSEÑAR

De cada uno: nombre, una línea de qué hace, y si puedes decir el nombre del cliente o hay que anonimizarlo («una gestoría de Barcelona»). **Con uno vale**. Una web con un caso bien contado convence más que una con seis fichas vacías que ponen «próximamente».

ANTES DE PUBLICAR EL NOMBRE DE UN CLIENTE, PÍDELE PERMISO

Aunque el trabajo sea tuyo y estés orgulloso de él. Es cortesía elemental, evita un disgusto y casi siempre dicen que sí — a menudo con una frase de recomendación de propina, que vale más que la ficha.

B) CÓMO QUIERES QUE TE CONTACTEN

¿Un correo `hola@apps4all.app` ? ¿WhatsApp? ¿Un formulario?

Mi recomendación para empezar: correo a secas. Un formulario en hosting compartido necesita programación por detrás, y los correos que genera suelen acabar en la carpeta de spam del destinatario — o sea, tú. Un enlace de correo funciona siempre, se lee en el móvil y no falla nunca. Cuando la web reciba visitas de verdad, ponemos formulario con un servicio que lo hace bien.

C) LOS DATOS DEL AVISO LEGAL

Nombre o razón social, NIF, domicilio y correo de contacto.

Por qué hace falta: en España, la ley LSSI-CE obliga a que toda web de una actividad económica identifique a quien está detrás. No es opcional y no es papeleo decorativo: es de las pocas cosas de una web que pueden acarrear una multa. Se resuelve con una página que se escribe una vez y no se toca más.

Lo que haré yo

En cuanto tengas hechas las tareas 1, 2 y 3, en una sesión de una o dos horas queda montado esto:

	Qué	Qué significa para ti
1	La estructura de carpetas y el primer commit	El proyecto pasa a estar bajo control de versiones. A partir de ahí, nada se pierde.
2	La portada	Logotipo, el claim <i>Crear, añadir, mejorar</i> y tus tres líneas de trabajo — comercios, fiestas de pueblo y producto propio—, cada una con el 4 en su color.
3	Proyectos, contacto y los dos textos legales	Las cuatro páginas que hacen que una web parezca una empresa y no una prueba.
4	La documentación del repositorio	Un <code>README</code> que responde cinco preguntas, y un <code>decisiones.md</code> con tres líneas por decisión. Es lo que agradecerás dentro de seis meses, cuando el proyecto te suene a ajeno.
5	El robot de publicación	Escrito y listo; solo tendrás que pegar las tres contraseñas del FTP en su caja fuerte.
6	La comprobación	Abro la web en el navegador y la miro, en pantalla grande y en móvil, antes de darla por buena.

CÓMO QUIERO QUE TRABAJEMOS, YA QUE QUIERES APRENDER

No te voy a soltar los archivos hechos y ya está. Cada vez que escriba algo te explicaré **qué hace y por qué así**, en el momento de escribirlo, que es cuando se entiende. Y cuando haya que elegir entre dos caminos, te contaré los dos y te diré cuál recomiendo y por qué — no al revés.

Tú puedes parar en cualquier punto y preguntar «¿esto qué es?». No hay pregunta demasiado básica: las básicas son justamente las que sostienen todo lo demás.

El día después

Cuando la web esté publicada, el ciclo de trabajo normal es este, y cabe en tres líneas:

1. Cambias o añades algo (o me lo pides).
2. Se guarda con un *commit* que explica qué cambió.
3. Se sube, y el robot lo publica solo en menos de un minuto.

Eso es todo. Sin ZIP, sin panel de control, sin miedo a romper nada — porque si se rompe, se vuelve a la versión anterior.

Por qué he elegido lo que he elegido

Las decisiones que tomé ayer, con su razón. Están cerradas a propósito para no volver a discutir las cada semana, pero **no están grabadas en piedra**: si alguna no te convence, la cambiamos ahora, que es cuando sale barato.

1 · HTML y CSS a mano, sin framework

La razón inmediata: en tu ordenador no hay Node ni npm instalados. Empezar con Astro serían dos o tres horas de instalación y configuración *antes* de escribir la primera línea que se pueda ver. Y tú has dicho justo lo contrario: prioridad a tener algo.

La razón de fondo, que pesa más: sin framework no hay paso de compilación, y eso significa que **lo que ves en tu carpeta es exactamente lo que hay en el servidor**. Cuando algo falla, hay un sitio donde mirar en lugar de tres. Para una web de cinco páginas, un framework resuelve un problema que todavía no tienes.

Qué se pierde: hay que repetir la cabecera y el pie en cada archivo. Con cuatro páginas es un incordio menor. Con diez, empieza a molestar — y ese momento es exactamente la señal para dar el paso, ni antes ni después.

Lo importante: este camino *no* es un callejón sin salida. Astro se traga el HTML que ya existe casi sin tocarlo.

2 · GitHub desde el primer día, aunque trabajes solo

La tentación es decir «esto es una prueba, ya lo subiré». Las pruebas se convierten en producción con una frecuencia sospechosa, y entonces el historial de sus primeros meses no existe.

Los tres motivos concretos están en el capítulo 2: poder deshacer, tener copia de seguridad de verdad, y que publicar sea gratis en esfuerzo. Añado un cuarto: es la misma herramienta que usarás en las apps Flutter. Aprenderla aquí, con una web donde no se puede romper nada grave, es el mejor sitio para aprenderla.

3 · Repositorio privado

Parece contradictorio —el contenido de la web va a ser público de todas formas— pero el repositorio guarda más que el HTML final: borradores, notas, textos a medio escribir, nombres de clientes antes de tener su permiso. Privado por defecto, y se abre el día que haya una razón concreta para abrirlo.

4 · Publicar a mano la primera vez, automatizar después

Podríamos montar el automatismo desde el principio. No lo hacemos por un motivo de método: **si montas dos cosas a la vez y falla, no sabes cuál de las dos ha fallado**. La primera subida manual sirve para confirmar que el dominio y el certificado funcionan. Con eso demostrado, el automatismo se monta sobre terreno firme y, si algo falla, ya sabes que es el automatismo.

Es la misma lógica de no cambiar dos piezas del coche a la vez.

5 · Sin analítica en la primera versión

Google Analytics pone cookies, y las cookies obligan por ley a un banner de consentimiento. Un banner es más trabajo, más código y una molestia para el visitante. Con cero visitas, medirlas no aporta nada.

Cuando de verdad importe saber de dónde llega la gente, hay herramientas que miden sin cookies — Plausible, Umami— y por tanto sin banner. Esa es la puerta correcta cuando llegue el momento.

6 · Cada página en su carpeta

Un detalle técnico que parece minucia y no lo es. Poniendo el archivo en `proyectos/index.html` en vez de `proyectos.html`, la dirección queda `apps4all.app/proyectos/`.

La ventaja aparece dentro de un año: cuando pasemos a Astro, **las direcciones no cambian**. Y las direcciones que no cambian conservan su posición en Google y no rompen los enlaces que alguien te haya puesto. Cambiar direcciones después de publicar es de las pocas cosas caras de deshacer en una web.

UN CRITERIO GENERAL QUE EXPLICA TODAS LAS DE ARRIBA

Aburrido gana a listo. La solución ingeniosa que no se entiende al releerla cuesta más de lo que ahorró. Cuando dudes entre dos opciones, elige la que hará el proyecto más predecible, no la más elegante — sobre todo sabiendo que quien lo va a mantener eres tú dentro de seis meses, y para entonces esto te sonará a escrito por otro.

Cómo crecerá la web

Aquí está la respuesta a tu preocupación de fondo: empezar sencillo *sin* que lo sencillo sea un callejón sin salida. Este es el camino, escalón a escalón. La regla es que **cada escalón se sube cuando el anterior se ha quedado corto de verdad**, nunca por adelantado.

Cuándo	Qué se hace	Cuesta
Ahora	Portada, contacto y aviso legal. Publicado.	Una sesión
Cuando tengas dos o tres trabajos que enseñar	Página de proyectos, una ficha por trabajo. Sigue siendo HTML suelto.	Un rato por ficha
Cuando repetir la cabecera en seis archivos te moleste	Aquí entra Astro. Se instala Node, y Astro absorbe el HTML que ya existe: la cabecera y el pie pasan a un molde común y las páginas se quedan como están. El robot de publicación cambia en una línea.	Media jornada
Cuando quieras escribir con regularidad	Un blog o unas notas. Astro lo trae de serie: escribes en texto plano y él hace las páginas.	Un par de horas
Cuando alguien pida presupuesto por la web	Formulario de contacto de verdad, con un servicio externo que se encarga del envío.	Una hora
Cuando importe de dónde llega la gente	Analítica sin cookies, y por tanto sin banner.	Media hora
Cuando tengas una app propia que enseñar	Una sección por producto: <code>apps4all.app/habitos/</code> . La estructura ya lo aguanta sin tocar nada.	Según la app

LA ÚNICA FORMA SEGURA DE NO TENER WEB NUNCA

Rehacer la versión 1 antes de publicarla. Es un patrón conocido y engancha, porque siempre hay algo mejorable y siempre parece razonable arreglarlo «antes de enseñarlo». Pero una web sin publicar tiene exactamente el mismo valor que un proyecto en la cabeza: cero.

Si algo del diseño no te gusta, se publica igual y se arregla la semana siguiente — sobre algo que ya está vivo, que ya te puede traer un cliente y que ya demuestra que existes.

Las tres reglas que evitan casi todos los disgustos

1. **Ninguna contraseña dentro del proyecto.** Nunca, ni un momento. Lo que entra en el historial de Git se queda ahí aunque borres la línea después.
2. **Publicado gana a perfecto.** Ver regla de arriba.

3. **Lo que dejes a medias, déjalo escrito.** Un *issue* en GitHub, dos líneas: en qué punto está y cuál era el siguiente paso. Reconstruir el contexto perdido cuesta más que el trabajo que quedaba pendiente.

Si algo falla

Ordenado por lo que pasa de verdad, no por lo que es más grave. Los tres primeros cubren casi todo.

Lo que ves	Lo más probable	Qué hacer
Página en blanco o «no se puede acceder a este sitio»	El SSL todavía no está activo. Con un <code>.app</code> , sin certificado no hay web (capítulo 4).	hPanel → Seguridad → SSL. ¿Pone <i>Activo</i> ? Si no, esperar.
Sale una lista de archivos y carpetas en vez de la web	El <code>index.html</code> quedó dentro de una subcarpeta al descomprimir el ZIP.	En el Administrador de archivos, mover el contenido a la raíz de <code>public_html</code> .
El navegador dice que el dominio no existe	DNS aún propagándose, si el dominio venía de otra empresa.	Esperar. Puede tardar hasta 24 h y no hay forma de acelerarlo.
Sale la página de bienvenida de Hostinger	Quedó su <code>index.html</code> o <code>default.php</code> de ejemplo, y el servidor lo enseña antes que el tuyo.	Borrar esos dos archivos. La carpeta <code>.well-known</code> , si está, no se toca: es del certificado.
La web se ve, pero sin colores ni formato	El archivo CSS no se subió, o la ruta al CSS está mal escrita.	Comprobar que existe <code>assets/css/</code> en el servidor. Es cosa mía, avísame.
<code>git push</code> pide usuario y contraseña y no acepta la tuya	GitHub ya no admite la contraseña normal por línea de órdenes.	Debe abrirse una ventana del navegador para autenticarte. Si no se abre, avísame.
El robot de publicación sale en rojo en la pestaña Actions	Casi siempre, una de las tres credenciales de FTP mal copiada.	Volver a escribir los tres <i>secrets</i> , con cuidado de no dejar espacios al final.

LA REGLA DE LOS QUINCE MINUTOS

Si llevas un cuarto de hora atascado en lo mismo sin avanzar nada, **para**. Insistir con el mismo enfoque no funciona casi nunca; lo que funciona es cambiar de enfoque o preguntar. Apúntame qué has hecho y qué has visto exactamente, y lo miramos. Es información, no un fracaso: la mitad de las veces el problema resulta ser una espera disfrazada de error.

Chuleta de órdenes

Para consultar, no para memorizar. Todas se teclean en una terminal, dentro de la carpeta del proyecto. **No hace falta que las ejecutes tú:** en la sesión de mañana las lanzo yo explicando cada una. Están aquí para que sepas qué pasó y puedas repetirlo solo.

Empezar el repositorio

```
git init
```

«A partir de ahora, lleva la cuenta de los cambios de esta carpeta.» Crea una carpeta oculta `.git` donde vivirá todo el historial. Se hace **una sola vez** en la vida del proyecto.

```
git add -A
```

«Prepara todos los cambios para la próxima foto.» El `-A` es *all*: todo lo nuevo, lo modificado y lo borrado. Es un paso previo, todavía no guarda nada.

```
git commit -m "chore: estructura inicial del sitio"
```

«Haz la foto, y llámala así.» El texto entre comillas es el concepto del asiento. El formato `tipo(zona): qué` no es capricho — es lo que hace que dentro de un año encuentres un cambio buscando por palabras:

Tipo	Para qué	Ejemplo
feat	Algo nuevo	feat(proyectos): añadir ficha de la gestoría
fix	Arreglar algo roto	fix(css): claim cortado en móviles de 320 px
chore	Mantenimiento, sin cambio visible	chore: actualizar el aviso legal

Lo que no vale: «cambios», «actualización», «pruebas». Dentro de un año no significan nada y convierten el historial en ruido.

Subirlo a GitHub

Con GitHub CLI instalado, una sola orden crea el repositorio y lo sube:

```
gh repo create apps4all-web --private --source=. --remote=origin --push
```

Traducida trozo a trozo: crea un repositorio llamado `apps4all-web` (`--private`, privado) a partir de esta carpeta (`--source=.`, donde el punto significa «aquí»), apúntalo como destino habitual (`--remote=origin`) y súbelo ya (`--push`).

Sin GitHub CLI: se crea el repositorio vacío en `github.com/new` —sin marcar ninguna de las casillas de «Add a README», o el primer envío chocará— y luego:

```
git remote add origin https://github.com/TU-USUARIO/apps4all-web.git
git branch -M main
git push -u origin main
```

Línea por línea: la primera apunta tu carpeta al repositorio de GitHub; la segunda nombra `main` a la rama principal; la tercera sube todo por primera vez.

El día a día

```
git status
```

«¿Qué he tocado desde la última foto?» Es la orden más útil de todas y no cambia nada — se puede lanzar cuantas veces se quiera. Ante la duda, siempre esta.

```
git add -A
git commit -m "feat(contacto): añadir el correo de contacto"
git push
```

Las tres de siempre, en ese orden: preparar, guardar con su nota, subir. Con el robot en marcha, ese `push` es además el gesto de publicar la web.

```
git log --oneline
```

El historial, una línea por cambio. Aquí es donde se cobran los buenos conceptos de commit.

LO ÚNICO QUE CONVIENE NO HACER SIN PREGUNTAR

Cualquier orden que lleve `--force`, `reset --hard` o `clean -fd`. Son las tres que sí pueden borrar trabajo de verdad. El resto de Git, como mucho, se deshace.

apps4all — Crear, añadir, mejorar

Guía de puesta en marcha de apps4all.app · 26 de agosto de 2026

La versión técnica y abreviada de este documento está en `PROCEDIMIENTO.md`, en la carpeta del proyecto.